МДК 02.01. **Технология разработки программного обеспечения**

Современные технологии разработки и инструменты интеграции

Курс
Глава 4

https://elearning.academia-moscow.ru/shellserver?id=4158944&demo=1&module_id=853164#853164

Задания

- ✓ ***Инструменты разработки программного обеспечения***

Задания 1, 2,3,4

- ✓ ***Структура проекта. Понятие репозитория***

Задания 1, 2

- ✓ ***Виды, цели, уровни интеграции***

Задания 1, 2

- ✓ **Организация работы команды в системе контроля версий**
- ✓ ***Задания 1, 2,3***

Инструменты разработки программного обеспечения

Инструментальное программное обеспечение представляет собой программное обеспечение, предназначенное для использования в ходе

- ❑ проектирования,
- ❑ разработки и
- ❑ сопровождения программ.

Системы программирования предназначены для разработки новых программ на конкретном языке программирования и включают

- компиляторы,
- трансляторы,
- интерпретаторы,
- компоновщики,
- препроцессоры исходных текстов,
- отладчики,
- текстовые редакторы,
- специализированные редакторы исходных текстов,
- библиотеки подпрограмм,
- генератор документации, а также
- диалоговую среду,
- справочные службы,
- средства автоматизированного тестирования,
- системы управления версиями,
- парсеры и др.

Компиляторы — программы, переводящие текст программы на языке высокого уровня в эквивалентную программу на машинном языке или в объектный модуль. Компилятор создает законченный вариант программы на машинном языке, который потом и выполняется компьютером. Процесс работы компилятора называют компиляцией.

Трансляторы — программное обеспечение или инструментальные средства, выполняющие трансляцию программы.

Интерпретаторы — программы (иногда аппаратные средства), анализирующие команды или операторы программы и тут же выполняющие их.

Компоновщики — программы, которые производят компоновку, т.е. принимают на вход один или несколько объектных модулей и собирают по ним исполнимый модуль.

Препроцессоры исходных текстов — компьютерные программы, принимающие данные на входе и выдающие данные, предназначенные для входа другой программы, например такой, как компилятор.

Отладчик является модулем среды разработки или отдельным приложением, предназначенным для поиска ошибок в программе.

Текстовые редакторы — компьютерные программы, предназначенные для создания и изменения текстовых файлов, а также их просмотра на экране, вывода на печать, поиска фрагментов текста и т.д.
Специализированные редакторы исходных текстов — текстовые редакторы для создания и редактирования исходного кода программ. Специализированный редактор исходных текстов может быть отдельным приложением или быть встроен в интегрированную среду разработки (Integrated Development Environment — IDE).
Библиотеки подпрограмм — сборники подпрограмм или объектов, используемых для разработки программного обеспечения.

SDK (Software Development Kit) — комплект средств разработки, который позволяет специалистам по программному обеспечению создавать приложения для определенного пакета программ, программного обеспечения базовых средств разработки, аппаратной платформы, компьютерной системы, видеоигровых консолей, оперативных систем и прочих платформ. Поставщики SDK иногда подменяют термин Software в словосочетании Software Development Kit на более точное слово. Например, Microsoft предоставляет Driver Development Kits (DDK) для разработки драйверов устройств.
Генератор документации — программа или пакет программ, позволяющие получать документацию, предназначенную для программистов и (или) конечных пользователей системы, по комментариям исходного кода, составленного особым образом, по исполняемым модулям (полученным на выходе компилятора).

Основные элементы интегрированной среды разработки

Интегрированная среда разработки программного обеспечения (IDE) представляет собой систему различных программных и инструментальных средств, используемых разработчиками программных продуктов.

Интегрированная среда разработки имеет общую интерактивную графическую оболочку, поддерживающую выполнение всех основных функций жизненного цикла разработки.

Единая интерактивная оболочка обеспечивает вызов всех других компонентов, не выходя из среды, с широким использованием функциональных клавиш. Интегрированная среда разработки включает текстовый редактор, систему поддержки сборки, компилятор и (или) интерпретатор, отладчик, систему управления версиями исходных кодов и разнообразные инструменты для упрощения конструирования графического интерфейса пользователя, браузер классов, инспектор объектов и другие средства.

Текстовый редактор используют для набора и редактирования исходных текстов программ.

Система поддержки сборки обеспечивает компиляцию проектов из исходных кодов. Включает *компилятор* исходного кода и *компоновщик* объектных бинарных кодов в единый исполняемый код (загрузочный модуль). Компоновщик используется либо штатный, входящий в состав операционной системы, либо специфичный для данной среды.

Отладчик используется для отладки программ в среде с помощью типичного набора команд: точки останова, пошагового выполнения процедур (методов), визуализации значений переменных и прочее.

Профилировщик — инструмент для мониторинга активности пользователей, накопления и анализа статистических данных, полученных в результате выполнения программы под управлением интегрированной среды: число вызовов методов, используемый объем памяти и т.д.

Рефакторинг применяется для реорганизации кода в целях облегчения понимания работы программы. Изменения касаются только внутренней структуры программы, не затрагивают ее внешнее поведение.

Генератор тестов используется для тестирования модулей, методов или процедур с различными возможными сочетаниями значений аргументов.

Система управления версиями исходных кодов применяется для облегчения работы с изменяющейся информацией. Система управления версиями позволяет хранить несколько версий файлов исходных кодов проектов в среде при сопровождении программ.

Инструменты поддержки командной разработки программ необходимы для обеспечения жизненного цикла программы — формирования требований, составления спецификаций, проектирования, реализации, тестирования. Они применяются для распределения задач среди членов команды разработчиков, контроля выполнения заданий менеджером проекта.

Инструменты анализа кода — специальное программное обеспечение для статического и динамического анализа кода проекта.

Статический анализ кода проводится без реального выполнения исследуемых программ. Полученная в результате анализа информация позволяет выявить места, где возможно содержатся ошибки, либо математически доказать какие-либо свойства программы (например, соответствие поведения спецификации).

Динамический анализ кода — анализ программного обеспечения, выполняемый на компьютере. При динамическом анализе может потребоваться загрузка специальных библиотек, перекомпиляция программного кода. Для эффективности динамического анализа требуется достаточное количество входных данных, чтобы получить более полное покрытие кода.

Инструменты «запутывания» кода выполняют замену имен элементов кода — классов, методов, полей — на непонятные, «случайные», «запутанные» имена в целях затруднения изучения декомпилированного бинарного кода, для защиты от «взлома» кода злоумышленниками.

Типовые шаблоны кода и *механизм разработки расширений* используются для поддержки создания различных видов программных проектов и решений — консольных приложений, web-приложений и web-сервисов, мобильных приложений, облачных и др. Для каждой из этих разновидностей требуется разработка специфической структуры файлов исходного кода и конфигурационных файлов.

Поддержка ***моделирования структуры программ*** **на унифицированном языке моделирования UML** также является неотъемлемой частью современного программирования.

Интегрированные среды поддерживают использование языка UML в двух направлениях:

- генерация модели и соответствующей диаграммы по исходному коду и,
- наоборот, генерация исходного кода по разработанной модели.

Современные интегрированные среды автоматизируют создание различного рода проектов, предоставляя шаблоны исходного кода и генерируя автоматически необходимые для проекта конфигурационные файлы. В настоящее время при программировании, как правило, используют готовые шаблоны кода, так как очень легко забыть вручную создать тот или иной файл — неотъемлемую часть проекта — либо упустить из вида какой-либо важный фрагмент кода. Поэтому поддержка интегрированными средами различных видов проектов особенно важна. Кроме того, набор возможных видов проектов в современных интегрированных средах является расширяемым, т. е. разработчик может ввести в среду при необходимости новый вид проекта.

Виды, цели и уровни интеграции программных модулей

Задача интеграции программного обеспечения заключается в том, чтобы объединить системы, созданные на базе различных технологий, в единую среду.
Интеграция обеспечивает взаимодействие независимо спроектированных систем и неоднородных, независимых источников данных.

Существуют следующие уровни интеграции:

- интеграция на уровне представления;
- интеграция на уровне функциональности;
- интеграция на уровне данных;
- комплексная интеграция.

Интеграция на уровне представления чаще всего предполагает применение общего визуального интерфейса. В качестве web-интерфейса пользователя, как правило, используется web-браузер. Web-браузеры являются основным программным обеспечением, посредством которого пользователи получают доступ к системам, приложениям и данным, размещенным в Интернете.

Интеграция на уровне функциональности подразумевает обеспечение прямого доступа к бизнес-логике приложений. Взаимодействие может осуществляться через web-сервисы, технологии COM, DCOM, CORBA и др.

Web-сервис — программная система со стандартизированными интерфейсами, которая идентифицируется web-адресом.

Интеграция на уровне данных предполагает доступ к одной или нескольким базам данных различных удаленных приложений. Этот тип интеграции широко распространен в самых разнообразных системах. Источниками данных систем могут быть традиционные системы баз данных, поддерживающие различные модели данных (реляционные, объектные, объектно-реляционные, графовые и др.), web-сайты, файлы структурированных данных, электронные таблицы и др. Интеграция данных включает объединение данных, находящихся в различных источниках, и предоставление данных пользователям в унифицированном виде.

Комплексная интеграция включает все названные ранее типы интеграции, а также еще дополнительные возможности, используемые в зависимости от поставленных задач.

Организация работы команды в системе контроля версий

Система контроля версий (СКВ) необходима для организации корректной работы с изменяющейся информацией.
Такие системы позволяют хранить несколько версий одного и того же файла, при необходимости возвращаться к более ранним версиям, определять авторов изменений и т.д.
Наиболее широко СКВ используют при разработке программного обеспечения для хранения исходных кодов программ, но могут с успехом применяться и в других областях, где используется большое число непрерывно изменяющихся документов.

За время разработки и вообще за время своего существования программное обеспечение достаточно часто меняется. Важно иметь не только последнюю версию программных файлов, но и несколько предыдущих.
В простейшем случае можно просто хранить несколько вариантов файлов, копируя их в отдельную папку. Но тогда нужно хранить несколько практически идентичных копий, что требует повышенного внимания и легко может привести к ошибкам.
Поэтому были разработаны средства для автоматизации контроля версий.

Системы контроля версий позволяют:

- хранить несколько версий одних и тех же файлов;
- возвращаться к любым предыдущим версиям файлов, получать актуальные на данный момент версии файлов;
- запоминать произведенные изменения в файлах;
- согласовывать действия разработчиков одной команды — они могут работать каждый над своей частью проекта, не мешая друг другу и не нанося ущерба;
- вести журнал изменений, в который разработчики могут записывать пояснения о том, что и почему они изменили в данной версии программы;
- контролировать права доступа пользователей, разрешая или запрещая чтение или изменение данных в зависимости от того, кто запрашивает это действие.

Системы контроля версий могут быть различных типов: локальные, централизованные и распределенные.

- *Локальные* СКВ хранят все версии проекта на компьютере разработчика.

- *Централизованные* СКВ имеют единое хранилище файлов, управляемое специальным сервером, который и выполняет бóльшую часть функций по управлению версиями. Разработчик имеет доступ к нужной ему версии файла из хранилища. Файл может быть выбран по номеру версии, по дате создания и другим характеристикам. После внесения изменений новая версия также помещается в хранилище, при этом предыдущая версия не уничтожается, а тоже остается в хранилище и может быть получена в любое время.

Все разработчики в команде имеют доступ к централизованному хранилищу. Однако существуют недостатки такой организации работы. У разработчика нет своего локального репозитория, а есть только копия данных с сервера. Если сервер выходит из строя, то команда не может работать над проектом. Другим недостатком использования централизованной СКВ является сложность одновременного доступа нескольких разработчиков к одному ресурсу.

- В *распределенной* СКВ кроме репозитория на сервере у каждого разработчика есть свой репозиторий. Это позволяет быстро восстановить актуальное состояние разрабатываемого программного обеспечения.

В настоящее время существует довольно большое число СКВ с различными возможностями:

- ***Git — распределенная СКВ;***
- CVS (система одновременных версий), централизованная СКВ;
- Apache Subversion (SVN); Mercurial, распределенная СКВ.

CASE-средства

CASE (*Computer Aided Software / System Engineering*) — набор инструментальных средств и методов программной инженерии для проектирования программного обеспечения.

CASE-технология — совокупность методологий разработки и сопровождения программных средств и сложных систем, поддерживаемая комплексом взаимосвязанных средств автоматизации.

CASE-система — набор CASE-средств, имеющих определенное функциональное назначение и выполненных в рамках единого программного продукта.

CASE-средства — программные средства, поддерживающие сопровождение полного жизненного цикла программного обеспечения, а именно: анализ и формулировку требований, проектирование баз данных и приложений, генерацию кода, тестирование, обеспечение качества, управление конфигурацией и проектом.